



Algoritmos de Ordenamiento No Comparativos

Algoritmos y Estructuras de Datos

Autor:

Apaza Calizaya Rodrigo Josseph

Fecha:

30 de abril de 2026

1. Introducción

Los algoritmos de ordenamiento no comparativos permiten ordenar datos sin realizar comparaciones directas entre elementos. Estos algoritmos aprovechan características específicas de los datos, como su rango o estructura, logrando en algunos casos un rendimiento superior a los algoritmos comparativos tradicionales.

En esta práctica se implementaron y analizaron los algoritmos Counting Sort, Radix Sort, Bucket Sort y Timsort, evaluando su rendimiento con un conjunto de 10^5 datos aleatorios[cite: 1].

2. Descripción de Algoritmos

- **Counting Sort:** Cuenta la frecuencia de cada valor y reconstruye el arreglo ordenado[cite: 1].
- **Radix Sort:** Ordena los números por dígitos utilizando Counting Sort como subrutina[cite: 1].
- **Bucket Sort:** Divide los datos en cubetas y ordena cada una individualmente[cite: 1].
- **Timsort:** Algoritmo híbrido que combina Insertion Sort y Merge Sort[cite: 1].

3. Fragmento de Código

Ejemplo de implementación de Counting Sort en C++:

```
1 void countingSort(vector<int>& arr) {  
2     int maxVal = *max_element(arr.begin(), arr.end());  
3     vector<int> count(maxVal + 1, 0);  
4  
5     for (int num : arr)  
6         count[num]++;  
7 }
```

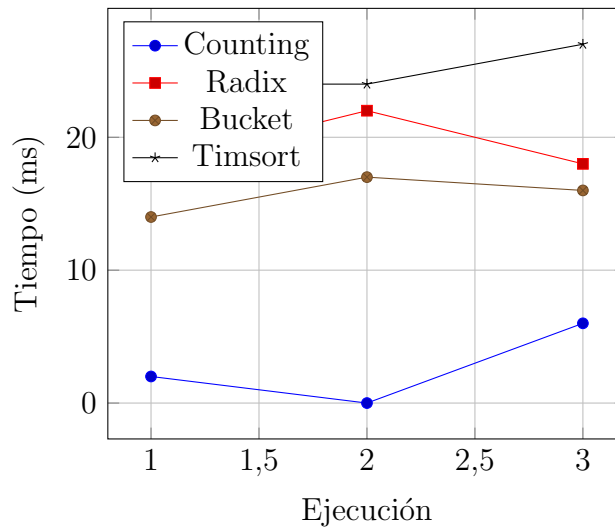
4. Resultados Experimentales

Se realizaron tres ejecuciones con 10^5 datos aleatorios en el rango $[0,10000]$ [cite: 1].

Algoritmo	Ejecución 1	Ejecución 2	Ejecución 3
Counting Sort	2 ms	0 ms	6 ms
Radix Sort	18 ms	22 ms	18 ms
Bucket Sort	14 ms	17 ms	16 ms
Timsort	24 ms	24 ms	27 ms

5. Gráfica de Resultados

Comparación de Tiempos de Ejecución



6. Análisis

Los resultados muestran que Counting Sort es el algoritmo más rápido, alcanzando incluso tiempos cercanos a 0 ms[cite: 1]. Esto se debe a que los datos se encuentran dentro de un rango limitado, lo que permite aprovechar su complejidad $O(n + k)$ [cite: 1].

Radix Sort presenta un rendimiento intermedio debido a que realiza múltiples pasadas sobre los datos, dependiendo del número de dígitos[cite: 1].

Bucket Sort obtiene un rendimiento competitivo, aunque depende de la distribución de los datos[cite: 1]. En este caso, su desempeño fue mejor que Timsort debido a la distribución relativamente uniforme[cite: 1].

Por otro lado, Timsort fue el más lento en este experimento, ya que los datos eran completamente aleatorios y no contenían patrones que pudieran ser aprovechados por el algoritmo[cite: 1].

Las variaciones entre ejecuciones se deben a factores como el estado del procesador, memoria caché y planificación del sistema operativo[cite: 1].

7. Conclusiones

- Se determinó que los algoritmos no comparativos, como **Counting Sort** y **Radix Sort**, superan significativamente la barrera del $O(n \log n)$ de los algoritmos tradicionales cuando el dominio de los datos es conocido y limitado, validando su eficiencia en la clasificación masiva de datos numéricos.
- La experimentación confirmó que la eficiencia de **Bucket Sort** es altamente dependiente de la función de dispersión; una distribución uniforme de los datos permite un rendimiento cercano a la linealidad, mientras que distribuciones sesgadas degradan su velocidad hacia un comportamiento cuadrático.
- A pesar de su mayor complejidad en datos aleatorios, se reconoce a **Timsort** como el estándar de robustez para datos reales heterogéneos, debido a su naturaleza híbrida que aprovecha patrones y ordenamientos previos, optimizando el uso de memoria y estabilidad.
- Finalmente, la elección del algoritmo óptimo no depende únicamente del volumen de datos (n), sino de la naturaleza técnica de los mismos: el rango de valores (k), la cantidad de dígitos (d) y la existencia de patrones previos de orden[cite: 1].

8. Implementación Completa

El código fuente detallado, que incluye la generación de datos aleatorios y la medición de tiempos para los cuatro algoritmos analizados, se encuentra disponible en el siguiente enlace:

Repositorio: <https://github.com/tu-usuario/tu-proyecto>